



Figure 3: Integration of Docker with generative AI based LLM models

following command:

```
CommandPrompt > ollama tune llama2
--dataset custom_dataset.json
```

Here, `custom\_dataset.json` represents the dataset that will be used to fine-tune the model. The tuning process enables the model to adapt to specialised tasks, improving its accuracy and relevance in generating outputs.

Docker integration of generative AI LLMs

Integrating generative AI and large language models with Docker involves a dedicated process that enhances the scalability, portability, and deployment efficiency of these advanced machine learning models. Docker, a containerisation platform, encapsulates applications and their dependencies into lightweight, self-sufficient containers, enabling

seamless deployment across various environments.

The first step in Dockerizing a generative AI LLM is to create a Docker image that contains the model, its runtime environment, and all requisite dependencies. This process typically begins with selecting a base image, such as an official Python image if the model is implemented in Python, or a specialised machine learning image like TensorFlow or PyTorch, depending on the framework used.

The Dockerfile, which serves as the blueprint for building the Docker image, is meticulously crafted to include the installation of the model's dependencies, such as specific Python libraries, CUDA drivers for GPU acceleration, and the model's codebase. For instance, installing large libraries like `transformers` for working with models like GPT or BERT, as well as configuring

environment variables for optimal runtime performance, are critical steps. Additionally, the Dockerfile should be optimised to minimise the image size, which can be achieved by leveraging multi-stage builds or minimising the number of layers.

Step 1: Installation and deployment of Docker

Download *Install Docker* for your operating system on the official Docker website.

Step 2: Create a Dockerfile

Create a file named Dockerfile in your project directory as follows:

```
# Use the official Ollama Docker image
FROM ollama/ollama
```

```
# Expose the Ollama API port
EXPOSE 11434
```

Step 3: Build the Docker image

Open your terminal, navigate to your project directory, and run the following command to build the Docker image:

```
docker build -t my-llm-image .
```

Replace *my-llm-image* with your desired image name.

Step 4: Run the Docker container

Run the following command to start a container from the image:

```
docker run -it -p 11434:11434 --name my-llm-container my-llm-image
```

- *-it*: This will run the container in interactive mode.
- *-p 11434:11434*: This will map port 11434 on your host to port 11434 in the container, allowing you to access the Ollama API.
- *--name my-llm-container*: This will give your container a name for easy management.
- *my-llm-image*: The name of the image you built in step 3.